颜色追踪

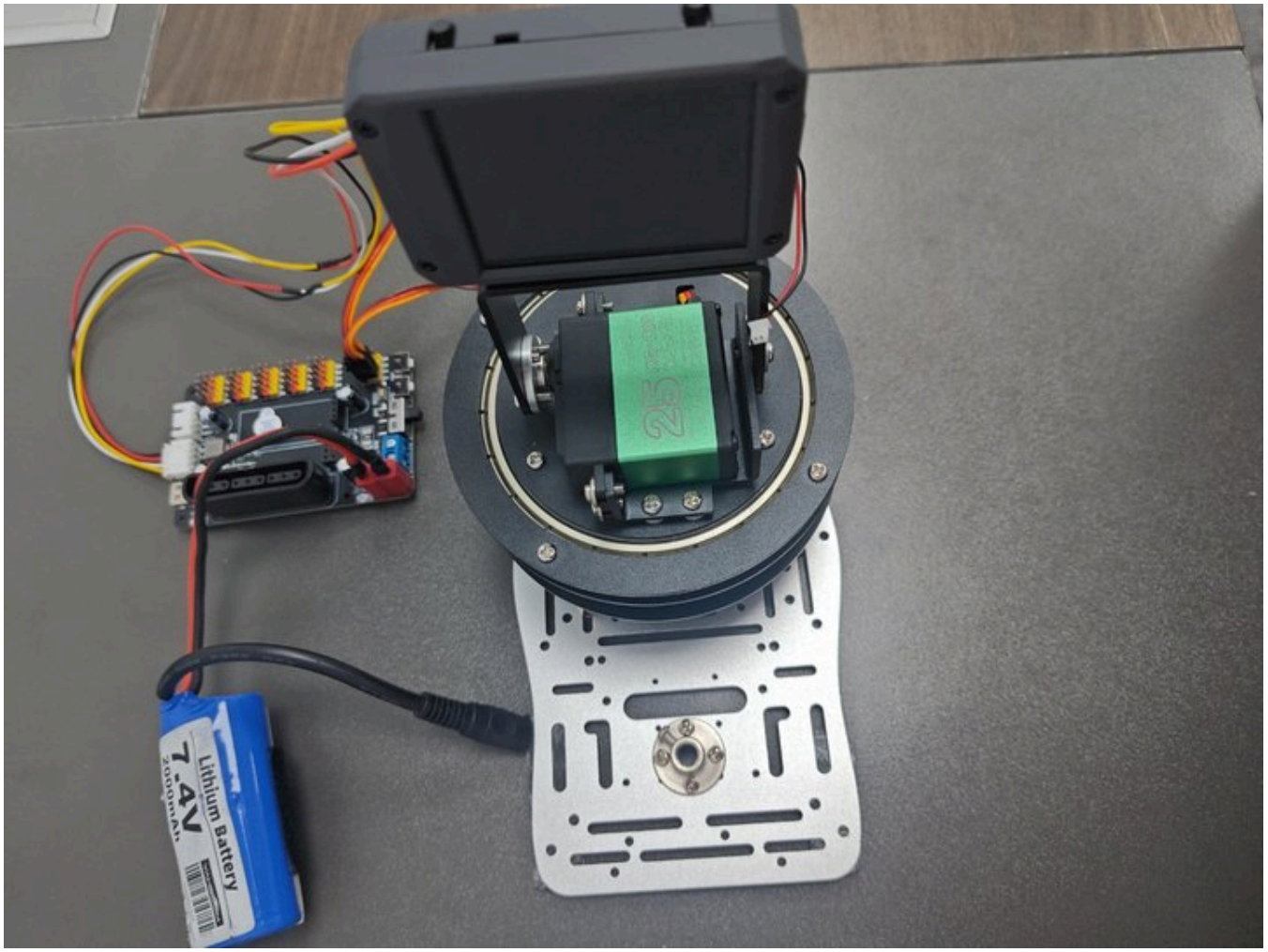
1.开篇说明

请先阅读二十四路舵机驱动板以及stm32入门教程里面的资料，了解清楚如何烧录程序、入门stm32的基础。这样方便学习和烧录后面的程序。

2.器材清单

- k230 x1
- 24路舵机驱动板 x1
- 7.4v电池 x1 购买
- 铝合金板 x1 购买

3.摆放位置

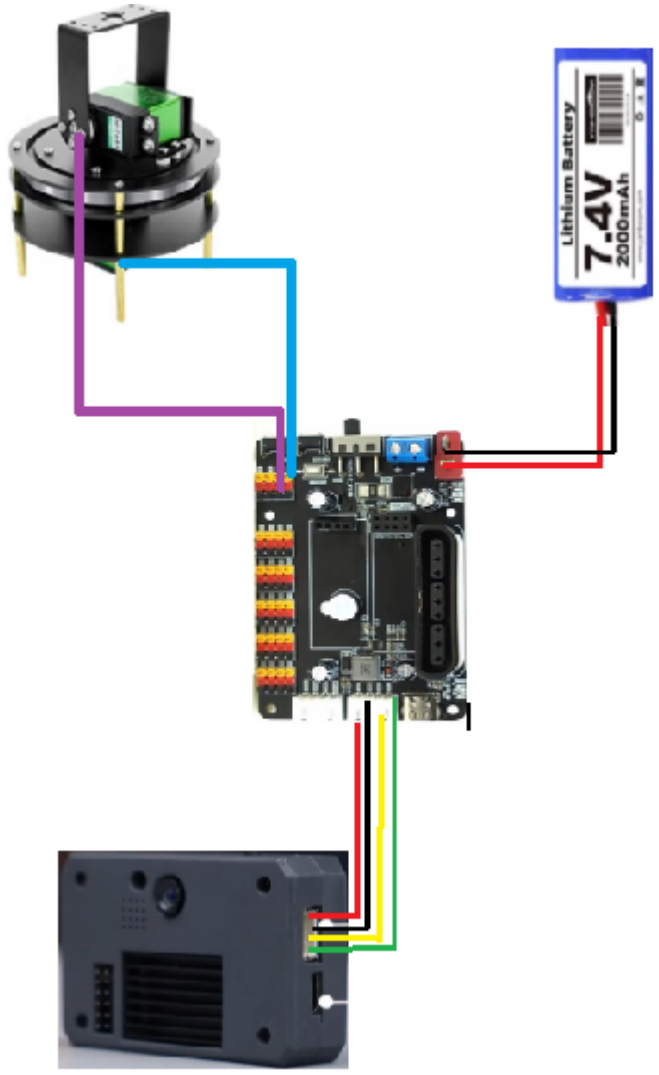


摆放位置如图所示，可以选择我们的铝基板车架作为固定k230和云台的底板，也可以自行选择能固定的底板，底盘可以胶给粘住以免云台动的幅度大导致偏移，k230固定在二维云台上面,用电池的时候如果云台转动幅度过大可以适度减小pid参数。

4.实验准备

首先按照二维云台安装教程搭建好二维云台

硬件接线：



二维云台	24路舵机驱动板
X轴舵机	S1
Y轴舵机	S2

K230视觉模块	24路电机驱动板
5V	5V
GND	GND
TX	RX3
RX	TX3

注意:插舵机的线时一定要将黄色线对准24路舵机驱动板的黄色GPIO口，褐色线对准24路舵机驱动板的黑色GPIO口。

5.K230代码解析

- **完整代码**

```
import time, os, sys
from media.sensor import *
from media.display import *
from media.media import *

from libs.YbProtocol import YbProtocol
from ybUtils.YbUart import YbUart
# uart = None
uart = YbUart(baudrate=115200)
pto = YbProtocol()

last_cx = 300 #初始点x坐标
last_cy = 220 #初始点y坐标
# 显示参数 / Display parameters
DISPLAY_WIDTH = 640   # LCD显示宽度 / LCD display width
DISPLAY_HEIGHT = 480  # LCD显示高度 / LCD display height

# LAB颜色空间阈值 / LAB color space thresholds
# (L Min, L Max, A Min, A Max, B Min, B Max)
THRESHOLDS = [
    (0, 66, 7, 127, 3, 127),     # 红色阈值 / Red threshold
    (42, 100, -128, -17, 6, 66),      # 绿色阈值 / Green threshold
    (43, 99, -43, -4, -56, -7),        # 蓝色阈值 / Blue threshold
    (37, 100, -128, 127, -128, -27)     # 亚博智能Logo的颜色 color of YAHBOOM
]

def get_closest_rgb(lab_threshold):
    """根据LAB阈值计算最接近的RGB颜色 / Calculate closest RGB color based on LAB threshold"""
    # 获取LAB空间的中心点值
    l_center = (lab_threshold[0] + lab_threshold[1]) // 2
    a_center = (lab_threshold[2] + lab_threshold[3]) // 2
    b_center = (lab_threshold[4] + lab_threshold[5]) // 2
    return image.lab_to_rgb((l_center,a_center,b_center))

def init_sensor():
    """初始化摄像头 / Initialize camera sensor"""
    sensor = Sensor()
    sensor.reset()
    sensor.set_framesize(width=DISPLAY_WIDTH, height=DISPLAY_HEIGHT)
    sensor.set_pixformat(Sensor.RGB565)
    return sensor

def init_display():
    """初始化显示 / Initialize display"""
    Display.init(Display.ST7701, to_ide=True)
    MediaManager.init()

def process_blobs(img, blobs, color):
```

```
    global last_cx
    global last_cy
    """处理检测到的色块 / Process detected color blobs"""
    for blob in blobs:
        img.draw_rectangle(blob[0:4], color=color, thickness=4)
        img.draw_cross(blob[5], blob[6], color=color, thickness=2)
        cx = blob[0]
        cy = blob[1]
        last_cx = cx#更新坐标
        last_cy = cy#更新坐标
        return cx, cy, 1
    return last_cx, last_cy, 0


def draw_fps(img, fps):
    """绘制FPS信息 / Draw FPS information"""
    img.draw_string_advanced(0, 0, 30, f'FPS: {fps:.3f}', color=(255, 255, 255))


def conversion_postion(x,y):
    if x > 510:
       x_h = 255
       x_l = 255
       x_z = x-255-255
    elif 510>=x>=255:
       x_h = 255
       x_l = x-255
       x_z = 0
    else:
       x_h = x
       x_l = 0
       x_z = 0
    if y > 510:
       y_h = 255
       y_l = 255
       y_z = y-255-255
    elif 510>=y>=255:
       y_h = 255
       y_l = y-255
       y_z = 0
    else:
       y_h = y
       y_l = 0
       y_z = 0
    return x_h,x_l,x_z,y_h,y_l,y_z

def main():
    try:
        # 初始化设备 / Initialize devices
        sensor = init_sensor()
        init_display()
        sensor.run()
```

```
        clock = time.clock()

        # 选择要检测的颜色索引 (0:红, 1:绿, 2:蓝) / Select color index to detect
        color_index = 1  # 可以修改这个值来选择检测不同的颜色
        threshold = THRESHOLDS[color_index]
        detect_color = get_closest_rgb(threshold)

        while True:
            clock.tick()
            img = sensor.snapshot()

            # 检测指定颜色 / Detect specified color
            blobs = img.find_blobs([threshold], area_threshold=5000, merge=True)
            if blobs:
                x,y,ok= process_blobs(img, blobs, detect_color)
                cx_h,cx_l,cx_z,cy_h,cy_l,cy_z = conversion_postion(x,y)
                uart.write(bytearray([0x55,0xaa,0x00,int(cx_h),int(cx_l),int(cx_z),0xfa]))
                uart.write(bytearray([0x55,0xaa,0xff,int(cy_h),int(cy_l),int(cy_z),0xfa]))
            fps = clock.fps()
            draw_fps(img, fps)

            print(fps)

            Display.show_image(img)

    except KeyboardInterrupt as e:
        print("用户中断 / User interrupted: ", e)
    except Exception as e:
        print(f"发生错误 / Error occurred: {e}")
    finally:
        if 'sensor' in locals() and isinstance(sensor, Sensor):
            sensor.stop()
        Display.deinit()
        MediaManager.deinit()

if __name__ == "__main__":
    main()
```

- **分割坐标函数**

```
def conversion_postion(x,y):
    if x > 510:
       x_h = 255
       x_l = 255
       x_z = x-255-255
    elif 510>=x>=255:
       x_h = 255
       x_l = x-255
       x_z = 0
    else:
```

```
        x_h = x
        x_l = 0
        x_z = 0
    if y > 510:
        y_h = 255
        y_l = 255
        y_z = y-255-255
    elif 510>=y>=255:
        y_h = 255
        y_l = y-255
        y_z = 0
    else:
        y_h = y
        y_l = 0
        y_z = 0
    return x_h,x_l,x_z,y_h,y_l,y_z
```

因为给stm32只是传输8位2进制数，所以最高传输数据为255，而我们K230的采用的是640x480因此需要对其坐标进行分割，将640拆成3帧数据进行传输。

- ## 代码结构

基本设置：

- 设定640×480分辨率的显示屏
- 使用LAB色彩空间定义了红、绿、蓝三种颜色的检测阈值
- DRAW_COLORS和COLOR_LABELS数组用于绘制标记和显示颜色标签

主要改进：

- 同时检测三种颜色
- 为检测到的物体添加颜色标签文字标注
- 简化了颜色处理逻辑

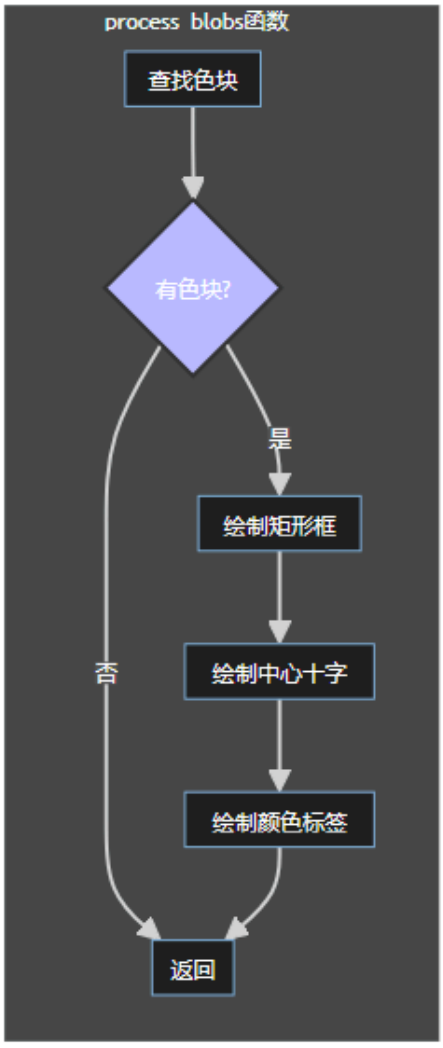
工作流程：

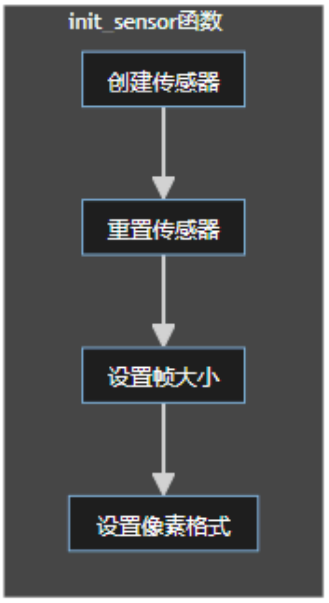
- 初始化摄像头和显示屏
- 进入主循环后：
    - 持续捕获图像
    - 依次检测三种颜色
    - 为每个检测到的物体绘制矩形框、十字标记和颜色标签
    - 显示帧率信息

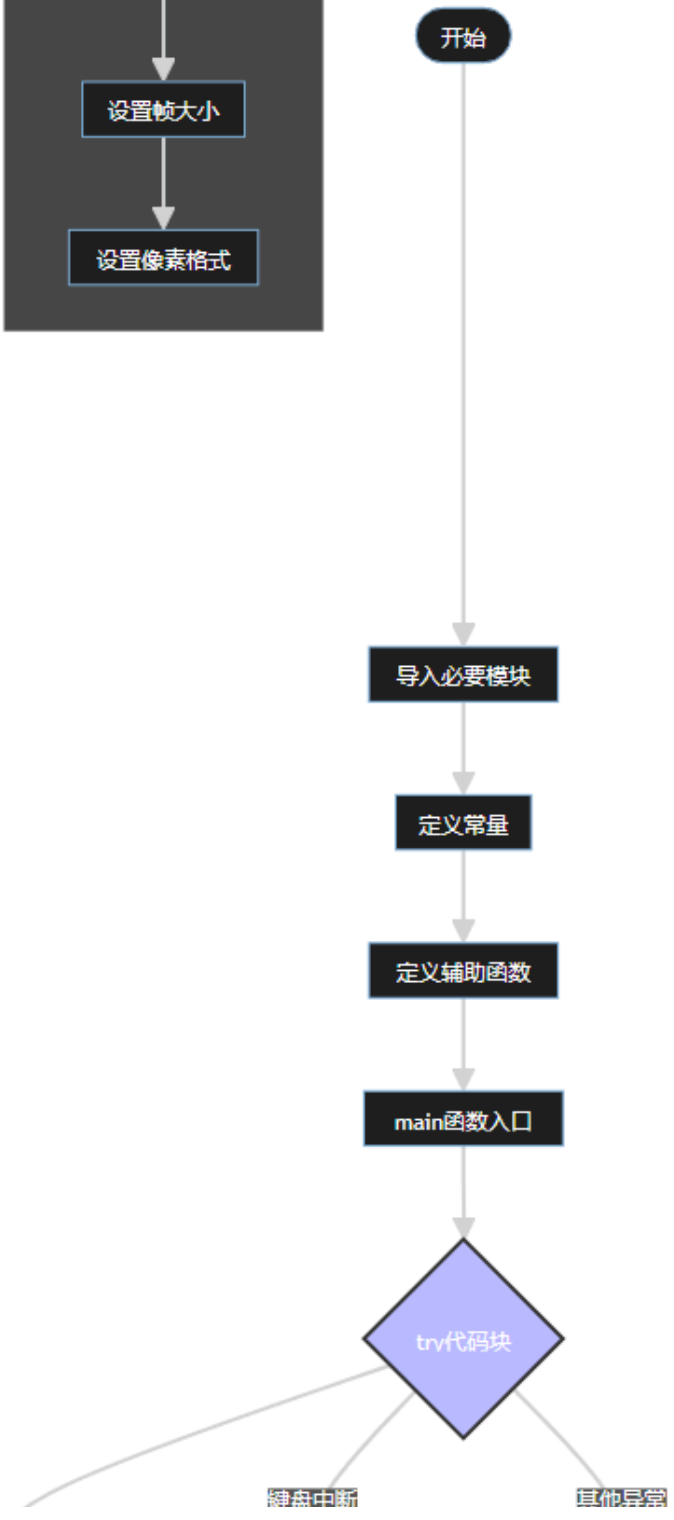
异常处理：

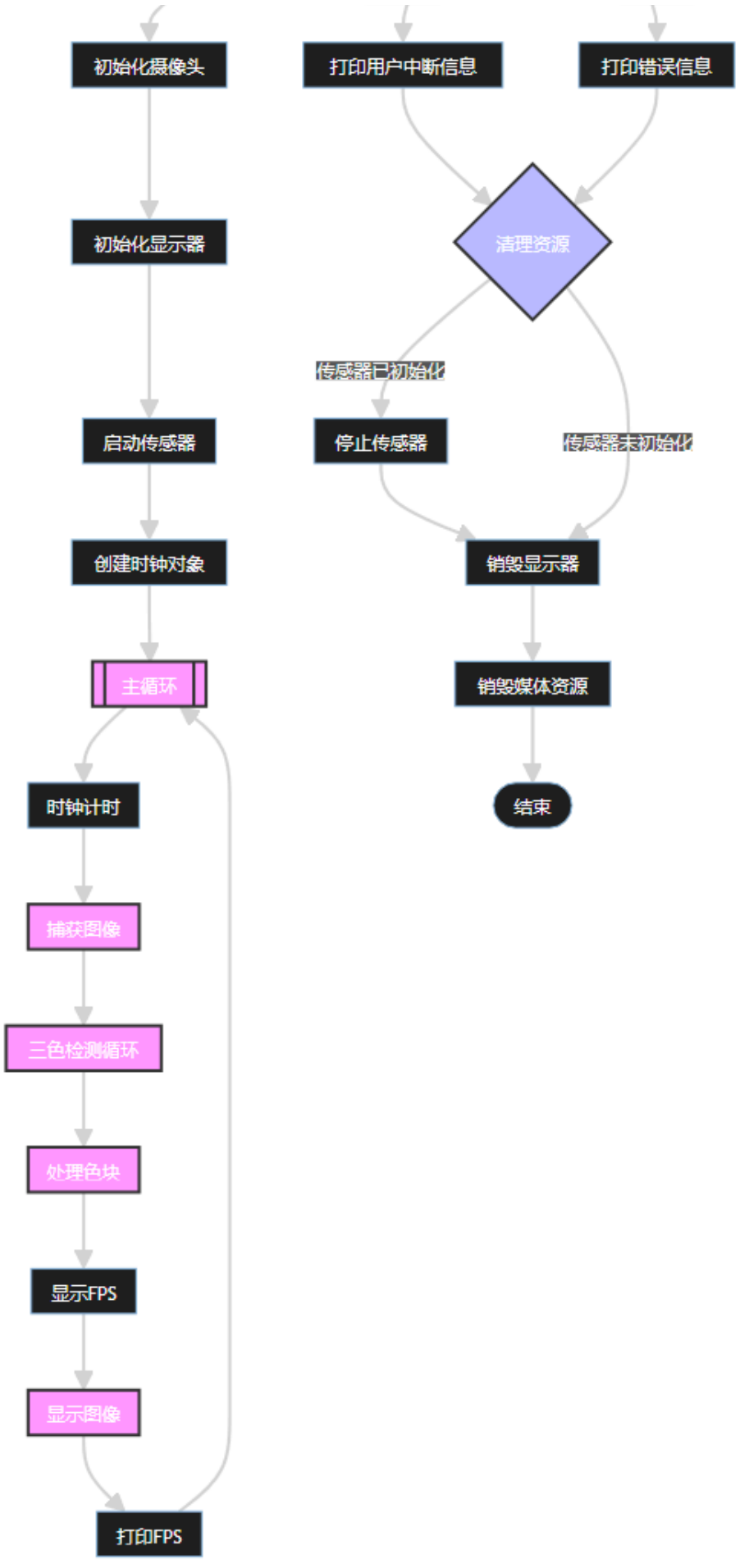
- 保留了完整的异常处理机制
- 确保程序结束时正确释放硬件资源

详细流程图如下：


process_blobs函数
查找色块
有色块?
是
否
绘制矩形框
绘制中心十字
绘制颜色标签
返回


init_sensor函数
创建传感器
重置传感器
设置帧大小
设置像素格式


开始
导入必要模块
定义常量
定义辅助函数
main函数入口
try代码块
键盘中断
其他异常


初始化摄像头
初始化显示器
启动传感器
创建时钟对象
主循环
时钟计时
捕获图像
三色检测循环
处理色块
显示FPS
显示图像
打印FPS
打印用户中断信息
打印错误信息
清理资源
传感器已初始化
传感器未初始化
停止传感器
销毁显示器
销毁媒体资源
结束

6.stm32关键代码解析

- ## control.c

```
uint8_t get_point2_p()    //处理坐标函数
{
    uint8_t lable, load_1, load_2 ,load_3 ,ret=0;
    if(DataDecode1(&Uart3_RingBuff,&lable, &load_1, &load_2,&load_3) == 0){
     if(lable == 0x00){
            rec_step_p[0][0] = load_1 + load_2 + load_3 ;
//          ret=1;

        } else if(lable == 0xff){

            rec_step_p[0][1] = load_1 + load_2 + load_3 ;
//          ret=1;

        }

    return ret;
}
}
```

此代码是将k230传过的3帧数据加在一起，由于前面说过将k230的坐标数据拆成3帧数据然后分别发送，这里正是将数据加在一起这样形成正确的坐标，将坐标赋值给二维数组。

```
    get_point2_p();

    set_target(rec_step_p[0][0],rec_step_p[0][1]);
```

此代码是在上面获得人脸坐标的基础上将坐标作为pid目标值，然后控制舵机运动。

- ## bsp_servo.c

```
ServTypdef Xserv = {
    .pwm = 2000,
    .MAXPWM = 2500,
    .MINPWM = 500,
    .MIDPWM = 1400
};

ServTypdef Yserv = {
    .pwm = 1000,
    .MAXPWM = 2300,
    .MINPWM = 500,
    .MIDPWM = 1000
};
```

此代码是设置舵机启动的起始位置，所以需要修改这个来拟合云台运动的顶点，我将云台开机初始位置设置在右上角，然后云台可以通过k230的坐标变化来按比例进行调整，如果在调整云台位置和细调位置的时候就需要调这个位置参数了。

```
void servo_ctr(ServTypdef *servo, int PidInput)
{
    uint16_t CompSet = servo->MIDPWM + PidInput;

    if( CompSet > servo->MAXPWM ){
        CompSet = servo->MAXPWM;
    } else if ( CompSet < servo->MINPWM  ){
        CompSet = servo->MINPWM;
    }
    if(servo->pwm == CompSet){/*避免重复更改比较值*/

        return;
    }

    if( servo == &Xserv ){


    GPIO_SetBits(Servo_J1_PORT, Servo_J1_PIN );      //将舵机电平置高
    delay_us(CompSet);                    //延时脉冲值的微秒数

    GPIO_ResetBits(Servo_J1_PORT, Servo_J1_PIN );            //将舵机电平置低
    delay_ms(20 - CompSet/1000);
    }
    else if( servo == &Yserv ){


    GPIO_SetBits(Servo_J2_PORT, Servo_J2_PIN );      //将舵机电平置高
    delay_us(CompSet);                          //延时脉冲值的微秒数

    GPIO_ResetBits(Servo_J2_PORT, Servo_J2_PIN );            //将舵机电平置低
    delay_ms(20 - CompSet/1000);
    }
}
```

此代码是将pid的输出给pwm，是将pid与pwm连接的地方，同时给pwm进行限幅，也是舵机运动的核心代码，通过延时和pwm的改变来控制舵机转动，将舵机运动和pwm联系一起。

- **pid.c**

```
float pid_calculate(PID_TypeDef *PID,float CurrentValue)
{
  PID->Err =  PID->Target - CurrentValue;
    PID->Integral += PID->Err;
    if(PID->Integral > 7000){
        PID->Integral = 7000;
    }
    if(PID->Integral < -7000){
```

```
        PID->Integral = -7000;
    }    //积分限幅
    PID->PID_out = PID->Kp * PID->Err                                    /*比例*/
         + PID->Ki * PID->Integral                                  /*积分*/
             + PID->Kd * (PID->Err - PID->LastErr);                     /*微分*/

    PID->LastErr = PID->Err;
        return PID->PID_out;
}
```

此代码是位置式pid运算的配置，包括积分限幅，位置式PID是通过计算当前偏差值（目标值与实际值之差）、积分值（过去误差的总和）和微分值（误差变化率），然后将这三个部分加权相加得到PID输出，如果想微调代码想达到更好的效果可以微调主函数的pid参数。

- **bsp_usart.c**

```
uint8_t DataDecode1(RingBuff_t *ringbuff, uint8_t *data1, uint8_t *data2, uint8_t
*data3,uint8_t *data4)
{
    static uint8_t uart_dec_count;
    static uint8_t uart_rec_data[6];
    uint8_t ret = 1;

    if(Read_RingBuff(ringbuff, &uart_rec_data[uart_dec_count]) == RINGBUFF_ERR){
        return 1;

    }
        if((uart_dec_count == 0)&&(uart_rec_data[uart_dec_count] != 0x55)) {            //帧
头0x55
        uart_rec_data[uart_dec_count] = 0;
    }
        else if((uart_dec_count == 1)&&(uart_rec_data[uart_dec_count] != 0xaa)){      //第二
帧0xaa
        uart_rec_data[uart_dec_count] = 0;
        uart_rec_data[uart_dec_count-1] = 0;
        uart_dec_count = 0;
        }else if((uart_dec_count == 6)&&(uart_rec_data[uart_dec_count] != 0xfa)){      //帧
尾0xfa
        uart_rec_data[uart_dec_count] = 0;
        uart_rec_data[uart_dec_count-1] = 0;
        uart_rec_data[uart_dec_count-2] = 0;
        uart_rec_data[uart_dec_count-3] = 0;
        uart_rec_data[uart_dec_count-4] = 0;
        uart_rec_data[uart_dec_count-5] = 0;
        uart_dec_count = 0;
    }
    else{
        if(uart_dec_count == 6)//成功接收一帧数据
        {
            *data1 = uart_rec_data[2];
        *data2 = uart_rec_data[3];
                *data3 = uart_rec_data[4];
```

```
            *data4 = uart_rec_data[5];
        ret = 0;

        }
        uart_dec_count++;
        if(uart_dec_count == 7)
        {
            uart_dec_count = 0;
        }

    }
    return ret;

}
```

此代码是接收k230数据的代码，在识别到帧头为0x55，第二个数据为0xaa，帧尾为0xfa的时候才接收数据，通过三个数据定位一帧数据，由于从k230传输过来的4个顶点坐标每个顶点又通过拆分成3组数据传输过来，所以传输过来的数据是非常庞大的，为了确保传过来的数据是准确的，所以采用三个数据来确认一帧数据。

7.效果描述

代码设定三种颜色的阈值，通过修改0.1.2来选择识别哪种颜色，当识别到选定颜色的时候，k230将坐标传输给二维云台，然后二维云台跟着颜色运动，从而达到追踪的效果。